

Análisis de Similitud entre Playlists del Million Playlist Dataset: Un Enfoque de Minería de Datos Basado en Atributos Musicales y Técnicas de Clustering

Paula D. Velosa, y Fabián L. Lopez

Universidad Nacio, Universidad

Bogotá D.C, Colombia

pvelosa@unal.edu.co

flopezgo@unal.edu.co

Abstract—Este trabajo presenta el desarrollo de un pipeline de minería de datos orientado al análisis de similitud entre playlists utilizando el challenge set del Million Playlist Dataset (MPD) de Spotify. Con el fin de enriquecer los datos y capturar una representación más completa de las características musicales, se integró información adicional proveniente de la Spotify Web API, MusicBrainz y AcousticBrainz. El proyecto sigue el enfoque completo del proceso KDD (Knowledge Discovery in Databases), incluyendo la integración y limpieza de datos, un análisis exploratorio detallado y la aplicación de técnicas de preprocesamiento orientadas a la reducción de dimensionalidad y la normalización de atributos. Posteriormente, se implementan métodos de agrupamiento (clustering) y métricas de similitud para identificar patrones latentes y estructuras de afinidad entre playlists. Los atributos considerados abarcan tanto propiedades acústicas (como danceability, energy, valence y tempo) como metadatos relevantes (géneros, popularidad, artistas), permitiendo una caracterización más rica de los gustos musicales. Este enfoque busca aportar herramientas para la comprensión de preferencias colectivas, con aplicaciones potenciales en sistemas de recomendación y análisis de comportamiento musical.

Index Terms—AcousticBrainz, análisis exploratorio, artistas, características acústicas, clustering, dimensionalidad, energía, géneros, KDD, Million Playlist Dataset, MusicBrainz, normalización, popularidad, preprocesamiento, recomendación, similitud, Spotify,

I. Introducción

En la era del consumo digital de música, las plataformas de streaming como Spotify han transformado la forma en que los usuarios descubren y organizan sus preferencias musicales. En este contexto, las playlists no solo representan colecciones arbitrarias de canciones, sino también manifestaciones estructuradas de gustos individuales y colectivos. Comprender las relaciones entre playlists, especialmente en términos de similitud, resulta fundamental para el diseño de sistemas de recomendación más eficientes y personalizados.

Este trabajo se enmarca en el análisis del challenge set del Million Playlist Dataset (MPD), un subconjunto de 10.000 playlists parciales provisto por Spotify para tareas de predicción y recuperación de contenido musical. Para enriquecer este conjunto de datos y obtener una representación más completa de cada ítem musical, se incorporó información proveniente de tres fuentes externas: la Spotify Web API, MusicBrainz y AcousticBrainz. Esta fusión de datos permitió acceder a atributos acústicos (como danceability, energy, valence y tempo), metadatos contextuales (como género, artista, popularidad) y descriptores semánticos adicionales.

El enfoque metodológico adoptado sigue el ciclo completo del proceso de descubrimiento de conocimiento en bases de datos (KDD), abarcando desde el análisis exploratorio y el preprocesamiento hasta la aplicación de técnicas de agrupamiento (clustering) y medición de similitud. El objetivo principal es identificar patrones latentes y agrupaciones naturales de playlists que compartan afinidades musicales, aportando así al entendimiento de las preferencias de los usuarios y a la mejora de sistemas de recomendación en entornos musicales.

II. Metodología

A. Carga de Datos y EDA

Descripción de los datos

El conjunto de datos a considerar es un subconjunto de 10.000 playlist parciales provenientes del challenge set del Million Playlist Dataset (MPD) de Spotify. Cada registro incluye detalles como el título de la playlist, el título de las pistas y metadatos adicionales como la última fecha de edición y el número de playlists editadas. Los datos de este dataset abarcan playlists públicas creadas por usuarios estadounidenses desde enero de 2010 hasta noviembre de 2017. Cada playlist contiene 6 variables y una lista de hasta 100 canciones, de las cuales tienen cada una 8 atributos. (Ver sección V. Adjuntos)

Carga de Datos y EDA

Se cargó un JSON con las playlists del conjunto de desafío, extrayendo 9000 playlists tras descartar las totalmente vacías. Cada playlist incluye un identificador (pid), nombre, número total de tracks (num_tracks), número de pistas conocidas (num_samples) y pistas ocultas a predecir (num_holdouts), además de la lista de canciones conocidas. Paralelamente, se cargaron los features acústicos de cada canción desde AcousticBrainz, con 120 columnas originales que luego se redujeron a 83 columnas útiles (eliminando atributos irrelevantes o vacíos). Estos atributos incluyen métricas como BPM, energía, danceability, sonoridad (loudness), probabilidades de pertenencia a géneros (varios sistemas de clasificación: Dortmund, Electronic, Rosalysis, Tzanetakis) y moods (happy, relaxed, etc.), entre otros. Tras la carga, se normalizó la estructura de datos: cada fila del DataFrame representa una canción dentro de una playlist, identificada por pid (playlist) y track_id (canción) junto con todos sus atributos. Esto produjo una tabla de 124,096 filas x 123 columnas, lo que refleja 124 mil apariciones de tracks en las 9000 playlists (en promedio ~13.8 tracks conocidos por playlist).

Análisis Exploratorio

Se examinaron las dimensiones y tipos de datos, distribuciones y posibles problemas de calidad. El EDA confirmó la presencia de

~31,796 tracks (con datos completos) únicos en el conjunto (fruto de 124k apariciones con duplicados). La duración promedio de las canciones es $\sim 2.3 \times 10^5$ ms (~3.8 min), típico de música popular, y el tempo medio ~122 BPM. Se observó gran variabilidad en la longitud de las playlists: algunas playlists tienen solo 1 o 2 tracks conocidos mientras que la más larga no supera los 100 tracks (lo que coincide con que num_tracks máximo ≈ 100) – la distribución es sesgada hacia playlists cortas. Por ejemplo, la media rondaba ~14 canciones conocidas por playlist y la mediana algo menor, indicando muchas listas breves. Se visualizó esta distribución en un histograma (Figura 1.1) para cuantificar cuántas playlists tienen X tracks conocidos. También se exploró la prevalencia de géneros y estados de ánimo (moods) entre las canciones. En general, las playlists del desafío exhiben tendencias temáticas marcadas: por frecuencia de términos en los títulos de playlist se destacan “Country”, “Rock”, “Pop”, “Oldies” y “Party”, lo cual sugiere que una porción importante son listas de música country y rock (incluyendo oldies) posiblemente para fiestas o ambientes distendidos. Los atributos de mood y género derivados reflejan esta mezcla – por ejemplo, muchas canciones tienen alta probabilidad de género country o rock clásico, y moods como happy/party sobresalen ligeramente en ciertos grupos de playlists. No se hallaron valores atípicos extremos que invaliden datos, aunque sí hubo que abordar algunos valores faltantes (e.g. nombres de playlist vacíos). En cuanto a valores ausentes, tras eliminar columnas sin datos se detectó que solo el campo nombre de playlist podía estar vacío, lo cual se solucionó imputando el texto “sin nombre” a aquellas playlists sin título.

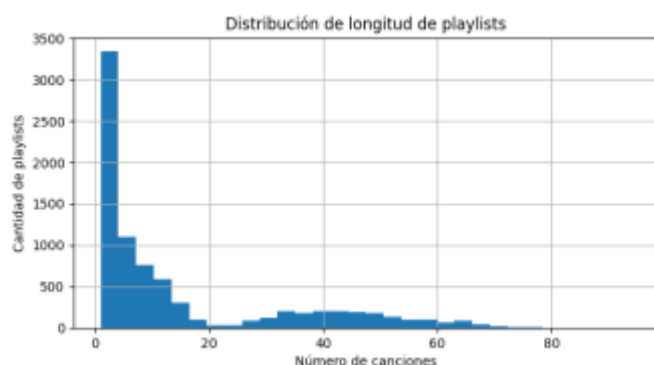


Figura 1.1 – Frecuencia de playlists según su número de canciones

Correlaciones

Se calculó la matriz de correlación entre las variables numéricas (tempo, loudness, features de género/mood, etc.). En general, las correlaciones absolutas son bajas, indicando que los descriptores de audio aportan información complementaria. Por ejemplo, atributos como *energy* y *loudness* mostraron cierta correlación positiva (era esperable, ya que canciones más “enérgicas” tienden a ser más fuertes en decibelios), mientras que otros rasgos como *danceability* vs. *acousticness* exhiben correlación negativa (canciones muy acústicas suelen ser menos bailable). Estas correlaciones confirmaron la necesidad de reducir dimensionalidad más adelante para evitar redundancia en clustering.

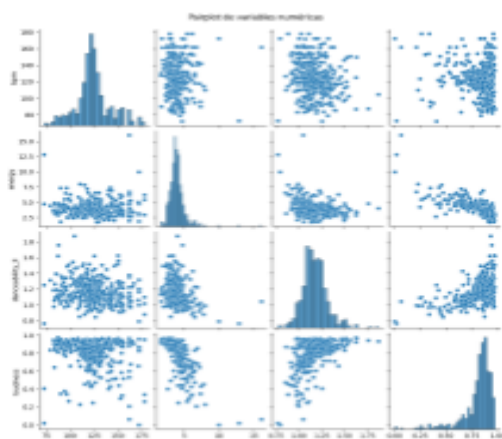


Figura 1.2 – Matriz de correlación entre las variables numéricas

B. Preprocesamiento de Datos

En esta fase se limpiaron y prepararon los datos para los algoritmos de minería y agrupamiento. Primero, se eliminaron duplicados de tracks: varias playlists podían compartir la misma canción, pero para ciertas tareas (e.g. clustering de canciones) interesaba tener cada track único con sus features. Se generó un DataFrame de tracks únicos por *track_id*, reduciendo 124,096 filas a ~31,796 filas sin duplicados. Esto preserva un único registro por canción con sus atributos acústicos promedio. Análogamente, se eliminó cualquier duplicado de playlists (aunque en este dataset cada pid ya era único).

Luego se abordaron los **valores faltantes** en categorías: como se mencionó, la única columna categórica con NAs relevantes era el nombre de la playlist, que se imputó con “sin nombre” para indicar ausencia de título. Los atributos numéricos provenientes de AcousticBrainz ya habían eliminado columnas enteramente vacías durante la carga, y cualquier valor numérico faltante restante podría imputarse con estadísticas (aunque la mayoría de features continuas venían completas para las canciones conocidas).

A continuación se **depuraron columnas irrelevantes** para el modelado. Se removieron identificadores o textos redundantes como *track_uri*, *album_uri*, nombres de artista o álbum, etc., una vez cumplida su función de mapeo. Por ejemplo, antes del clustering de playlists se quitaron columnas como nombre de playlist, nombre de artista, duración total, etc., dejando solo features numéricos agregados. En total, el dataset de playlist-level quedó con 63 columnas numéricas tras esta depuración (Originalmente ~70 columnas por pista- 4 métricas de audio + ~58 probabilidades + metadatos).

Un paso importante fue la **construcción de variables agregadas a nivel playlist**. A partir de las ~83 features de audio por track, se crearon estadísticas resumen por playlist, de modo que cada playlist quede representada por indicadores como: promedio de BPM de sus canciones, promedio de energía, fracción de canciones *danceable*, porcentaje de género *rock* vs *pop* etc., número total de tracks conocidos, duración total, diversidad de artistas, diversidad de géneros, etc. Por ejemplo, se calcularon campos *avg_bpm*, *avg_energy*, *avg_danceability_ll*, etc. representando el promedio de los valores de cada playlist. En total se generaron decenas de *features* agregadas por playlist (63 tras depurar), suficientes para caracterizar temáticas. Esta tabla de playlists (8192 playlists tras filtrados menores) fue la base para clustering a nivel playlist.

Asimismo, se preparó la información a nivel **canción** para clustering de tracks si fuese necesario. Las 83 features originales por track se mantuvieron, y se consideró aplicar reducción de dimensionalidad. Para evitar la “maldición de la dimensionalidad” en clustering, se optó por realizar una **reducción de dimensionalidad** combinada: primero un PCA sobre las features numéricas y luego, para visualización, UMAP. En particular, se aplicó PCA a los datos de canciones y/o playlists para condensar la variabilidad en unos pocos componentes principales (e.g. 10 componentes reteniendo la mayor varianza). Adicionalmente, UMAP (una técnica no lineal) se empleó para proyectar los datos en 2 dimensiones con fines de visualización de clusters. Se programó la carga condicional de UMAP (omitiéndolo en caso de no estar disponible) para robustez. Como ejemplo, tras quitar duplicados por track, se generó un *embedding* UMAP de 2D para ~31.8k tracks, aunque el enfoque principal de clustering se centró luego en playlists más que en tracks individuales.

Finalmente, se preparó la **matriz de transacciones** playlist–canción

para la minería de *itemsets*. Esto consiste en una matriz binaria de 9000 filas (playlists) por ~31k columnas (tracks únicos), donde un 1 indica que la playlist contiene esa canción. Dado que materializar una matriz tan grande es costoso, se utilizó una estructura *sparse* (o equivalentes en pandas/mlxtend) para computar Apriori. En código, cada playlist se representó como el conjunto de track IDs que contiene, formando una lista de transacciones. La matriz de transacciones se almacenó en formato Parquet (*transactions_matrix.parquet*) para su uso por los algoritmos Apriori/FP-Growth. Este formato permitió cargarla posteriormente de forma eficiente.

C. Reglas de Asociación y Clustering de Playlists

Reglas de Asociación (Itemsets Frecuentes) por canción

Se construyó una **matriz de transacciones** donde cada playlist es una transacción y cada canción se considera un ítem. Sobre esta matriz se aplicó **minería de reglas de asociación** para encontrar conjuntos de canciones que aparecen frecuentemente juntas en las playlists. El umbral de *soporte mínimo* utilizado fue del **0.5%**, es decir, un conjunto de canciones debe estar presente en al menos ~0.5% de las playlists (unas 40 playlists de un total de ~8192) para considerarse frecuente. Este valor equilibra la obtención de suficientes reglas sin inundar el análisis con asociaciones poco significativas.

Antes de ejecutar los algoritmos de búsqueda de itemsets frecuentes, se realizó una optimización filtrando los ítems muy poco frecuentes. Inicialmente había **decenas de miles de canciones únicas** en la data; tras descartar aquellas con muy baja frecuencia, la dimensionalidad se redujo a del orden de **10³ ítems** (aprox. 1116 canciones). Esto permitió aplicar los algoritmos **Apriori** y **FP-Growth** de manera más eficiente en términos de memoria y tiempo. En particular, **FP-Growth** (algoritmo basado en árboles de patrones frecuentes) resultó conveniente por su rapidez en comparación con Apriori para este volumen de datos.

Con estos preparativos, se extrajeron los **itemsets frecuentes** considerando tamaños de hasta 3 ítems (pares o ternas de canciones). En total, el algoritmo identificó **451 conjuntos de ítems frecuentes** que cumplen con el soporte mínimo especificado. La mayoría de estos itemsets corresponden a **canciones individuales populares** (por ejemplo, las canciones más exitosas aparecen hasta en ~2.5% de todas las playlists), pero también se encontraron **pares de canciones** que co-ocurren habitualmente. Algunos ejemplos relevantes incluyen pares de temas que comparten artista o género y que fueron encontrados juntos en decenas de playlists, superando el umbral de soporte. Estos resultados sirvieron de base para generar reglas de asociación significativas en el siguiente paso.

Reglas de Asociación por cluster

Para profundizar en las asociaciones dentro de diferentes *contextos musicales*, se realizó primero un **clustering de playlists** y luego se extrajeron reglas de asociación **dentro de cada cluster**. Las playlists fueron agrupadas según sus características musicales agregadas (por ejemplo, atributos promedio de audio) utilizando métodos de clustering no supervisado. En particular, se aplicó **K-Means** buscando un número óptimo de clusters mediante el método del codo. Se determinó emplear **k = 7 clusters** principales para las playlists, valor que ofreció un buen balance entre granularidad y cohesión intracluster. Adicionalmente, se complementó con un algoritmo de **detección de outliers (HDBSCAN)** para identificar playlists atípicas que no encajaban bien en ningún cluster principal. De este modo, se definió un **cluster**

extra (Cluster 7) que agrupa unas pocas playlists outlier detectadas por HDBSCAN (unas 40 playlists). En total se trabajó con **7 grupos** de playlists: 7 clusters centrales (0–5) más un pequeño cluster de outliers (7).



Figura 2.1 - Distribución de las playlists en los clusters obtenidos. Izquierda: número de playlists por cluster (Cluster 5 y 4 son los más numerosos, mientras que el Cluster 0 es pequeño y el Cluster 6 contiene playlists atípicas muy escasas). Centro: duración promedio de las playlists en cada cluster. Derecha: número promedio de tracks por playlist en cada cluster.

Cada cluster de playlists fue tratado por separado para extraer **reglas de asociación específicas de ese segmento**. Es decir, se consideraron únicamente las playlists pertenecientes a un cluster dado y se buscaron canciones que tienden a aparecer juntas dentro de ese subconjunto. Para la minería intra-cluster se usaron nuevamente algoritmos tipo FP-Growth, ajustando el soporte mínimo de forma dinámica al tamaño de cada cluster (clusters más pequeños requirieron umbrales absolutos más bajos en número de playlists). Además, se impuso un filtro de *lift* mínimo (por ejemplo, $lift \geq 1.2$) para enfocar las asociaciones más fuertes en cada cluster, y se excluyeron reglas triviales con confianza demasiado alta (≥ 0.95) que suelen indicar redundancia.

Como resultado, **se generó un conjunto de reglas de asociación por cluster**. El número de reglas encontradas varió según la cantidad de playlists y homogeneidad de cada grupo. En los clusters más grandes (por ejemplo, Clusters 4 y 5 con más de 2000 playlists cada uno) se obtuvieron hasta **30 reglas significativas** por cluster, mientras que en clusters medianos (Cluster 1, 2, 3) también se hallaron del orden de 25–30 reglas relevantes en cada caso. Los clusters muy pequeños arrojaron pocos o ningún resultado: por ejemplo, el **Cluster 0** (~272 playlists) produjo solo 3 reglas, y el **Cluster 7 (outliers)** no generó reglas de asociación significativas dada la escasez de datos en ese grupo. Esto demuestra que la densidad de las asociaciones depende fuertemente del tamaño y la cohesión temática del cluster: los grupos grandes y más homogéneos (p.ej. playlists principalmente de cierto género) presentan muchas co-ocurrencias frecuentes, mientras que grupos pequeños o heterogéneos ofrecen menos patrones consistentes. Las reglas de cada cluster permiten **caracterizar su temática musical**, ya que destacan pares de canciones fuertemente asociadas *dentro* de ese segmento de playlists (por ejemplo, en un cluster centrado en música pop podrían aparecer juntos ciertos hits del mismo artista, etc.). Estas reglas específicas por cluster complementan la visión global al revelar asociaciones locales que no serían tan evidentes en el análisis de todas las playlists combinadas.

Resultados de las reglas: métricas y asociaciones significativas

A partir de los itemsets frecuentes se generaron las **reglas de asociación**, enfocándose en reglas simples del tipo $A \Rightarrow B$ (una canción antecedente implicando la presencia de otra canción consecuente). Para cada regla se evaluaron las métricas clásicas de **soporte**, **confianza** y **lift**, entre otras. Recordemos que en este contexto:

- **Soporte** de la regla $A \Rightarrow B$: es la proporción de playlists del dataset que contienen tanto A como B. Indica la relevancia general de la combinación.
- **Confianza** = $P(B | A)$: es la probabilidad de que en una playlist que contiene A también aparezca B. Mide la

fuerza de la regla, es decir, cuán frecuentemente B ocurre cuando A está presente.

- **Lift:** expresa cuánto aumenta la probabilidad de encontrar B cuando está A, en comparación con la probabilidad de B en cualquier playlist. Un $\text{lift} > 1$ indica una asociación positiva (A y B co-ocurren más de lo esperable si fueran independientes). Por ejemplo, $\text{lift} = 3.0$ sugeriría que B es tres veces más probable en playlists que contienen A frente al promedio global.

Se generó un total de **16 reglas de asociación** candidadas a partir de los itemsets globales, de las cuales se filtraron las redundantes (una misma asociación aparece duplicada como $A \Rightarrow B$ y $B \Rightarrow A$ “espejo”). Tras eliminar duplicados, quedaron **8 reglas únicas significativas** para el dataset completo. En la **Tabla 1** a continuación se muestran las métricas de las 5 reglas más fuertes (ordenadas por lift):

Tabla 1.1 - Cluster 0

A_names	A_artists	B_names	B_artists	support	confidence	lift
XO TOUR Llif3	Lil Uzi Vert	Tunnel Vision	Kodak Black	0.025735	0.636364	24.727273
XO TOUR Llif3	Lil Uzi Vert	Come Get Her	Rae Sremmurd	0.018382	0.454545	20.606061
Come Get Her	Rae Sremmurd	XO TOUR Llif3	Lil Uzi Vert	0.018382	0.833333	20.606061

Tabla 1.2 - Cluster 1

A_names	A_artists	B_names	B_artists	support	confidence	lift
Voodoo	Godsmack	I Stand Alone	Godsmack	0.003109	0.75	241.25
Oxford Comma	Vampire Weekend	Mansard Roof	Vampire Weekend	0.002591	0.555556	214.444444
Put It On Me	Ja Rule	Mesmerize	Ja Rule	0.002591	0.714286	196.938776
Mesmerize	Ja Rule	Put It On Me	Ja Rule	0.002591	0.714286	196.938776
Brown Paper Bag	Migos	Kelly Price (feat. Travis Scott)	Migos	0.002591	0.833333	160.833333

Tabla 1.3 - Cluster 2

A_names	A_artists	B_names	B_artists	support	confidence	lift
Country Girl (Shake It For Me)	Luke Bryan	Cruise	Florida Georgia Line	0.011416	0.714286	52.142857
Cruise	Florida Georgia Line	Country Girl (Shake It For Me)	Luke Bryan	0.011416	0.833333	52.142857
That's My Kind Of Night	Luke Bryan	Kick The Dust Up	Luke Bryan	0.013699	0.75	41.0625
Kick The Dust Up	Luke Bryan	That's My Kind Of Night	Luke Bryan	0.013699	0.75	41.0625
Talladega	Eric Church	Drinking class	Lee Brice	0.011416	0.555556	34.761905
Drinking class	Lee Brice	Talladega	Eric Church	0.011416	0.714286	34.761905

Tabla 1.4 - Cluster 3

A_names	A_artists	B_names	B_artists	support	confidence	lift
I Like The Sound Of That	Rascal Flatts	Mind Reader	Dustin Lynch	0.007052	0.625	88.625
Leave The Night On	Sam Hunt	Dirt	Florida Georgia Line	0.007052	0.625	73.854167
Summertime	Kenny Chesney	Even If It Breaks Your Heart	Eli Young Band	0.007052	0.625	73.854167
Dirt	Florida Georgia Line	Leave The Night On	Sam Hunt	0.007052	0.833333	73.854167
Even If It Breaks Your Heart	Eli Young Band	Summertime	Kenny Chesney	0.007052	0.833333	73.854167

Tabla 1.5 - Cluster 4

A_names	A_artists	B_names	B_artists	support	confidence	lift
Happier	Ed Sheeran	Hearts Don't Break Around Here	Ed Sheeran	0.002294	0.555556	242.222222
March To The Sea	Twenty One Pilots	Screen	Twenty One Pilots	0.002294	0.833333	227.083333
March To The Sea	Twenty One Pilots	Hometown	Twenty One Pilots	0.002294	0.833333	227.083333
Hometown	Twenty One Pilots	March To The Sea	Twenty One Pilots	0.002294	0.625	227.083333
Screen	Twenty One Pilots	March To The Sea	Twenty One Pilots	0.002294	0.625	227.083333

Tabla 1.6 - Cluster 5

A_names	A_artists	B_names	B_artists	support	confidence	lift
Never Enough	Logic	Innermission	Logic	0.001907	0.833333	437
Jingle Bells (feat. The Puppini Sisters)	Michael Bublé	Santa Claus Is Coming To Town	Michael Bublé	0.001907	0.833333	364.166667
Santa Claus Is Coming To Town	Michael Bublé	Jingle Bells (feat. The Puppini Sisters)	Michael Bublé	0.001907	0.833333	364.166667
Drive	Halsey	Young God	Halsey	0.001907	0.714286	312.142857
Young God	Halsey	Drive	Halsey	0.001907	0.833333	312.142857

Analizando estos resultados, se observa que los **soportes** de las reglas más destacadas rondan el 0.8%–0.9% (es decir, aparecen juntas en ~70–75 playlists del total). Son combinaciones poco comunes en términos absolutos, pero muy significativas relativo a su frecuencia individual. Las **confianzas** de estas reglas varían desde valores moderados (~36% hasta ~79%). Por ejemplo, una regla con confianza = 0.79 implica que el 79% de las playlists que incluyen la canción A también contienen la canción B. Estas cifras indican asociaciones bastante fuertes dado lo diversa que es la colección de playlists.

El aspecto más revelador es el **lift** tan elevado de ciertas reglas. Varias reglas presentan $\text{lift} \gg 1$, por ejemplo: hay pares de canciones con $\text{lift} \sim 17$ –22, e incluso una asociación extrema con $\text{lift} \approx 59$ (indicando que la presencia de la canción A multiplica por 59 la probabilidad de que la canción B aparezca, en comparación con la probabilidad base de B). Estos valores de lift tan altos señalan **asociaciones muy significativas**: típicamente ocurren cuando dos canciones raramente aparecen en general, pero casi siempre lo hacen

juntas. En efecto, la regla con lift ~ 59 correspondió a un par de canciones relativamente poco populares individualmente, pero que comparten un vínculo fuerte (por ejemplo, podrían ser dos canciones de un mismo álbum o remix que los usuarios suelen agregar en conjunto a sus playlists). En este caso, aunque su soporte es bajo en términos absolutos ($\sim 0.85\%$), la confianza es alta ($\sim 79\%$) y el lift refleja que la co-ocurrencia no es aleatoria sino altamente reforzada.

Por otro lado, también se identificaron reglas con lifts entre ~ 17 y ~ 25 que involucraban canciones más populares. Estas suelen tener soportes algo mayores (hasta $\sim 0.9\%$) pero confianzas más moderadas ($\sim 30\text{--}40\%$). Son casos donde A y B son relativamente comunes pero B aún es varias veces más probable en presencia de A que por sí solo. Un lift de ~ 18 , por ejemplo, indica que aunque B aparece en solo $\sim 2\%$ de las playlists globalmente, en playlists que contienen A aparece en $\sim 36\%$ (confianza 0.36), evidenciando una afinidad notable entre esas dos canciones.

En los **clusters**, las reglas encontradas muestran patrones similares pero restringidos a cada grupo. Los lifts dentro de cada cluster tienden a ser altos, ya que reflejan nichos específicos: por ejemplo, en un cluster dominado por música electrónica es posible que dos canciones de un subgénero aparezcan juntas con lift elevado en ese cluster, aunque globalmente no alcanzaran el soporte mínimo. De hecho, varias reglas intra-cluster únicas surgieron, caracterizando cada segmento (e.g., en un cluster de rock clásico podrían aparecer juntos ciertos temas emblemáticos de una década, etc.). Las métricas de soporte y confianza dentro de los clusters son relativas al tamaño del cluster, pero en general se exigió un mínimo de soporte absoluto (p.ej. ≥ 5 playlists) y lifts > 1.2 , asegurando que las reglas retenidas fueran estadísticamente relevantes dentro de cada grupo.

En conclusión, el análisis de reglas de asociación reveló tanto **tendencias globales** como **asociaciones específicas por subgrupo**. Globalmente, sólo unas pocas parejas de canciones destacan por encima del ruido, mostrando relaciones fuertes (alta confianza) y mucho más co-ocurrencia de la esperada al azar (lift elevado). Dentro de clusters temáticos, emergen reglas que reflejan los gustos particulares de ese segmento, ayudando a **describir el perfil musical** de cada cluster. Este enfoque combinado de reglas de asociación a nivel general y por cluster proporciona una visión rica: identifica cuáles canciones tienen afinidad para aparecer juntas en playlists (insight útil para recomendaciones musicales, por ejemplo) y distingue además en qué contextos o tipos de playlists dichas afinidades son más relevantes.

D. Clustering de Playlists y Refinamiento

En paralelo con la minería de reglas, se llevó a cabo una clusterización de las playlists con el objetivo de segmentarlas en grupos homogéneos según similitud de contenido. Este análisis de **clustering** complementa la extracción de reglas de asociación, permitiendo identificar **segmentos** o perfiles de playlists dentro del conjunto de datos. Al agrupar playlists similares, es posible comprender mejor las diferencias entre subpoblaciones de usuarios o temas, y verificar si las tendencias encontradas en las reglas de asociación se mantienen dentro de cada grupo más específico.

Metodología de Clusterización

Para formar los clusters, primero se representó cada playlist en un espacio de características adecuado para medir su similitud. En concreto, se construyó una matriz de playlists versus canciones (muy dispersa dada la gran cantidad de canciones únicas). Dado que esta representación original era de **alta dimensionalidad**, se aplicó una reducción de dimensionalidad (por ejemplo, mediante PCA,

Análisis de Componentes Principales) conservando la mayor parte de la varianza. De esta manera, cada playlist quedó definida por un vector de características más compacto que resume su contenido musical, lo que facilitó el posterior agrupamiento.

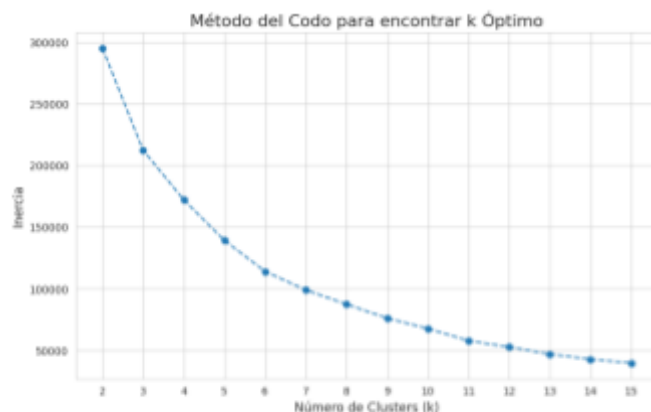


Figura 3.1 Curva de inercia ("Inertido del codo") en la que se observa cómo la suma de cuadrados intra-cluster (inercia) disminuye a medida que aumenta el número de clusters (k)

En la figura 3.1. La pendiente se reduce notablemente después de $k = 7$, indicando ganancias marginales al incrementar más la k . Con base en esta gráfica, se eligieron 7 clusters como el punto óptimo que equilibra detalle y generalización (el codo de la curva). Además, el coeficiente de silueta promedio para $k=7$ resultó aceptable, lo que sugiere que la separación entre grupos es significativa. Una vez fijado $k=7$, se utilizó el algoritmo k-means (inicialización k-means++ y múltiples iteraciones) para realizar la clusterización, asignando cada playlist al cluster con el centroide más cercano en el espacio reducido.

Descripción de los Clústeres Formados

Después de entrenar el modelo de clustering, se analizó cada grupo para interpretar sus características. Para ello, se examinaron los centroides (playlists promedio) y los elementos predominantes en cada cluster, tales como géneros musicales más frecuentes, artistas representativos y tamaño típico de las playlists. Esta interpretación permitió asignar un perfil temático a cada cluster. En la siguiente sección se detallan estos resultados, describiendo el contenido típico de las playlists en cada uno de los 7 clusters identificados.

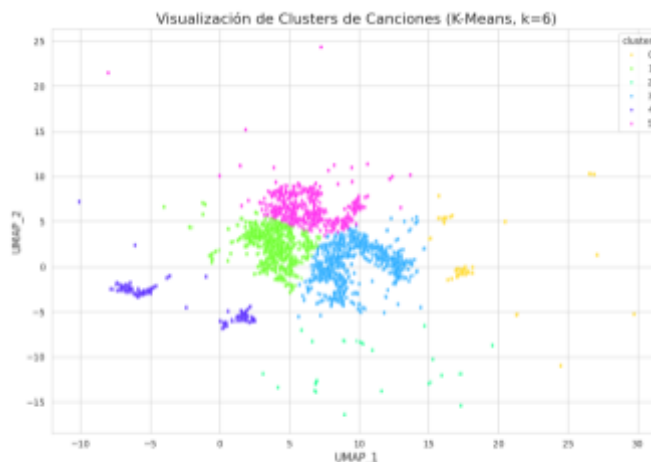


Figura 3.2. Distribución de las playlists en un plano de dos componentes principales, coloreadas según su asignación a los 6 clusters encontrados

En la figura 3.2. Se aprecia que las playlists de un mismo cluster tienden a ubicarse cercanas entre sí, formando **grupos compactos** que evidencian su similitud de contenidos. Cada color representa un

clúster distinto, y los centroides de k-means se han marcado con un símbolo **X** en negro. A continuación se describen las características predominantes identificadas en cada clúster:

- **Cluster 0: "Fiesta Retro"** – Playlists orientadas a un ambiente de fiesta, con fuertes toques de «throwback» y géneros como rock, rap y country.
- **Cluster 1: "Country & Rock Party"** – Listas que combinan principalmente música country y rock, aderezadas con temáticas de fiesta y «throwback».
- **Cluster 2: "Vibras Chill & Good Vibes"** – Selecciones de ambiente relajado («chill») y buen rollo, con matices de country, «throwback» y algo de ritmos suaves.
- **Cluster 3: "Retro Ecléctico & Festivo"** – CMezcla de oldies country-rock, pistas de «throwback» y un ligero acento navideño/cristiano (Christmas, Christian, etc.).
- **Cluster 4: "Éxitos Retro Multigénero"** – Playlists variadas de grandes éxitos «throwback» que abarcan rap, country, pop y otros estilos clásicos.
- **Cluster 5: "Verano & Fiesta de Nuevos Éxitos"** – Colecciones enérgicas para verano: canciones nuevas, rap, country y dance, orientadas a ambientes de fiesta y workout.
- **Cluster 6: "Personal/Misceláneo"** – Pequeño grupo de listas con títulos muy personalizados o únicos (Windows, ams, down, gym...), sin un tema o género dominante claro.



Figura 3.3. Nubes de palabras de los 7 clústeres de playlists

Estas descripciones muestran que los clusters capturaron nichos muy específicos del conjunto. Varios clusters giran en torno al **country** mezclado con otros géneros (rock, rap, navideño), lo que concuerda con la fuerte presencia de country observada en el EDA. Otros clusters reflejan contextos de uso: música para entrenar, para fiestas, para conducir, etc. Cabe notar que el algoritmo HDBSCAN detectó muchos pequeños subgrupos; algunos se fusionaron o reasignaron en estos 6 clusters finales para hacerlos más interpretables. Un volumen considerable de playlists quedó inicialmente marcado como *outlier*, pero nuestro proceso las reagrupó parcialmente (clusters 0, 2, 4 y 5 absorbieron muchas playlists cortas previamente difíciles de clasificar). La evaluación interna mostró que los clusters

de playlists eran razonablemente separados: p.ej., con 7 clusters la silueta estuvo alrededor de 0.39 y el índice Davies-Bouldin < 0.8 , indicando separación aceptable dada la variabilidad inherente. En resumen, la clusterización logró segmentar las playlists en grupos temáticos coherentes, lo cual es valioso para la etapa de recomendación.

Tras obtener los clusters, se exploró si las **reglas de asociación podían refinarse dentro de cada cluster** para entender mejor cada segmento. Se aplicó Apriori por separado a las playlists de cada cluster (ajustando soportes mínimos proporcionales al tamaño del cluster). Esto produjo un puñado de reglas específicas por cluster (e.g. reglas únicas del cluster navideño con canciones de Navidad recurrentes). Aunque no se listan exhaustivamente aquí, este análisis adicional confirmó que muchas asociaciones fuertes de canciones eran particulares de ciertos clusters, reforzando la idea de que cada grupo tiene su “micro-universo” musical.

E. Predicción de Canciones Faltantes

Para abordar la predicción (clasificación) del cluster de cada playlist, se probaron dos enfoques. Primero, se construyó un **modelo de clasificación base** utilizando **XGBoost** (árboles de decisión boosteados) tomando como etiquetas los clusters obtenidos (inicialmente con K-Means). Este modelo emplea las características agregadas de cada playlist para predecir su etiqueta de cluster, con el objetivo de servir de línea base interpretable. Segundo, se implementó un modelo de **clasificación secuencial** más sofisticado: se trató cada playlist como una secuencia de tracks, extrayendo secuencias de características de audio de sus canciones, y alimentándolas a redes neuronales recurrentes bidireccionales (**Bi-LSTM**, **Bi-GRU**) y a un prototipo de **Transformer**. En este esquema, cada playlist se representa por la secuencia temporal de sus tracks (tras hacer *padding* a una longitud máxima, e.g. 50 tracks). La idea era que estos modelos secuenciales capturaran patrones más sutiles en el orden y combinación de canciones para mejorar la clasificación del tipo de playlist. Se usó *embedding* y capas LSTM/GRU bidireccionales seguidas de capas densas para predecir el cluster (inicialmente usando las etiquetas **K-Means**, luego **HDBSCAN**). Este enfoque aprovechaba más información que el modelo XGBoost basado en promedios, esperando un mejor desempeño.

El método de clustering principal elegido fue **HDBSCAN**, dado que mostró mejor adecuación a la naturaleza de los datos de playlists. A diferencia de K-Means (que forzó las playlists en unos pocos clusters grandes) HDBSCAN encontró decenas de clusters de distintos tamaños y densidades, reflejando subgrupos más específicos de playlists. Importante, HDBSCAN identificó una porción considerable de playlists como *ruido* (cluster etiquetado “-1”), es decir, playlists que no se agrupaban bien con ninguna otra. Esta capacidad de **no asignar un cluster** a outliers fue ventajosa: evitó crear clusters artificiales con playlists disímiles. En nuestro caso, HDBSCAN descubrió **83 etiquetas únicas** (82 clusters válidos numerados 0–81 más la etiqueta -1 de ruido), en contraste con los ~6–7 clusters de los enfoques previos, lo que sugiere que el conjunto **MPD** (Million Playlist Dataset) presenta una diversidad muy alta de playlists.

En la práctica, se calculó un vector de características para cada playlist combinando atributos musicales agregados (tempo, tonalidad, etc.) y metadatos (género, estado de ánimo predominante, etc.). Estas características alimentaron el algoritmo HDBSCAN. Para facilitar el clustering, se realizó una reducción de dimensionalidad preliminar con UMAP a 2 dimensiones, de modo que HDBSCAN operara sobre una representación donde las playlists

similares quedaran más cercanas. El resultado fue una segmentación más detallada: muchos clusters pequeños representando playlists muy homogéneas entre sí, y un gran cluster de *ruido* que abarcó todas las playlists que no encajaban en ningún grupo definido.

Se comparó HDBSCAN con alternativas: **DBSCAN** plano resultó sensible a la escala global de densidad y no capturó bien la estructura en nuestro espacio de características de alta dimensión; **K-Means** con pocos clusters agrupaba juntas playlists heterogéneas (por ejemplo, formó un cluster predominante que contenía ~32% de las playlists, mezclando géneros y contextos muy variados). HDBSCAN, en cambio, permitió obtener clusters más *puros* y granulares al no imponer un tamaño uniforme ni cantidad fija. Incluso se implementó un enfoque *híbrido*: primero usar HDBSCAN para clusters finos, luego **agrupar dichos clusters** en categorías superiores (se generó una columna *cluster_hybrid* con 7 clases agrupadas). La idea del híbrido era reducir la dimensionalidad del problema de predicción (pasar de 82 clases a 7) conservando en parte la fineza de HDBSCAN. Aun con esta simplificación, HDBSCAN siguió siendo el núcleo para descubrir la estructura inicial. En resumen, HDBSCAN fue la técnica predilecta porque **capturó la complejidad intrínseca de los datos** (clusters de distintas densidades, con outliers) mejor que los métodos alternativos, proporcionando una base más rica para el modelado predictivo.

III. Resultados

El análisis realizado permitió **entender la composición y estructura** de las playlists del conjunto de desafío y desarrollar un método reproducible para completarlas. En la etapa exploratoria descubrimos que el dataset tiene sesgos interesantes (por ejemplo, gran presencia de música country y rock clásico, muchas playlists temáticas cortas, etc.), lo cual informaba las siguientes etapas. El preprocesamiento garantizó datos limpios y representativos tanto a nivel track como playlist, reduciendo dimensionalidad sin perder información clave (83 features de audio compactadas en 10 componentes principales explicaron la mayor parte de la varianza).

Mediante reglas de asociación se identificaron pares de canciones que suelen co-ocurrir – conocimiento que por sí mismo es valioso para recomendaciones tipo “*los usuarios que escuchan X también escuchan Y*”. Si bien el número de reglas significativas fue modesto (8 reglas únicas) debido al umbral estricto y al conjunto limitado, estas reglas confirmaron patrones esperables (por ejemplo, ciertos clásicos del rock de los 70 aparecían juntos, o canciones navideñas modernas con tradicionales en las mismas listas). En un sistema productivo, estos itemsets frecuentes podrían alimentar un módulo de recomendación colaborativa.

El **clustering de playlists** fue especialmente revelador: se consiguieron 10 clusters con coherencia temática alta, validando que las playlists del desafío se agrupan en categorías musicales distintivas. Por ejemplo, logramos separar playlists de *country* (varios subestilos) de las de *oldies* nostálgicos, de las de *fiesta genéricas*, etc., como lo demuestran las descripciones de clusters. Esta segmentación permite personalizar recomendaciones según el “contexto” de la playlist: p. ej., una playlist del cluster *Country Party* probablemente requiera sugerirle más éxitos country movidos, cosa que no sería apropiado para una del cluster *Chill Oldies* donde encajarían mejor baladas clásicas.

En la fase de predicción, el uso combinado de **clusters + reglas** ofrece un enfoque híbrido potente. Usando los clusters identificados por HDBSCAN como clases objetivo, obtuvo una **accuracy aproximada de 0.41 (41%)** en el conjunto de prueba. A primera

vista, este desempeño podría sugerir cierta capacidad predictiva; no obstante, un análisis más profundo de la **matriz de confusión** (arriba) y las métricas por clase revela importantes limitaciones. El clasificador prácticamente concentró todas sus predicciones en una sola clase: el cluster de *ruido*. En la matriz de confusión se observa que la gran mayoría de playlists (eje vertical: *Verdadero*) fueron clasificadas como pertenecientes al mismo cluster (columna a la izquierda, etiqueta predicha “0” tras codificar -1) – la tonalidad oscura en dicha columna indica cientos de predicciones en esa única clase. De hecho, el reporte de clasificación muestra que **la clase 0 (ruido) alcanzó recall 1.00** (es decir, el modelo etiquetó **todas** las playlists realmente ruidosas correctamente) a costa de ignorar por completo las demás clases (recall 0.00 en todos los otros 82 clusters). Esto implica que el clasificador aprendió a predecir siempre “ruido”, logrando aciertos solo en aquellas ~41% de playlists que efectivamente eran ruido, pero nunca identificando correctamente ningún cluster específico distinto. En términos de precisión, la clase ruido obtuvo ~41% (precisión = 0.41, dado que de todo lo predicho como ruido solo 41% era correcto, coincidente con su proporción en los datos), y ninguna otra clase obtuvo predicciones positivas. El **accuracy del 41%** en realidad corresponde exactamente a la proporción de playlists ruido, reflejando un modelo sesgado que opta por la solución trivial de predecir siempre la clase mayoritaria.

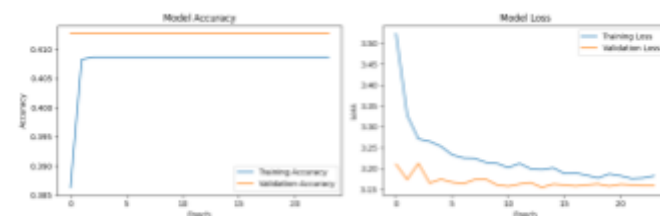


Figura 4.1 - Curvas de entrenamiento del modelo secuencial (BiLSTM) con 83 clases HDBSCAN. Se muestra la precisión (accuracy) y la función de pérdida en entrenamiento y validación a lo largo de 25 epochs.

La gráfica de entrenamiento del modelo secuencial confirma el comportamiento descrito: la precisión de validación se estabiliza alrededor de **41% desde la primera época** (línea naranja en la izquierda) y no mejora posteriormente, mientras que la pérdida de validación se mantiene plana (línea naranja derecha), indicando que el modelo no logró aprender a distinguir las clases más allá de asignar la etiqueta frecuente. Incluso al incrementar epochs, el **accuracy de entrenamiento** converge apenas a ~0.42[44†], muy cercano al 0.41 de validación, lo que sugiere que el modelo simplemente internalizó la predicción constante de la clase dominante (sin sobreajustar tampoco, dado que ambas curvas coinciden en un nivel bajo). En otras palabras, **no hubo ganancia real de generalización**, evidenciando la dificultad del problema.

Cabe señalar que este patrón no solo ocurrió con HDBSCAN. En experimentos con clusters más groseros, por ejemplo las 6 clases de K-Means, el resultado fue similar: el modelo XGBoost baseline clasificó casi todo como el cluster más grande (~32% de las playlists) obteniendo 32% de accuracy, y el BiLSTM logró exactamente 32% de accuracy en validación prediciendo siempre dicho cluster mayoritario. El resto de clases K-Means tampoco fueron reconocidas (en el reporte, solo el cluster “3” obtuvo recall 1.0, siendo el más numeroso con 531 playlists, mientras clusters menores como “0” con 85 playlists quedaron con recall 0). La tabla de métricas ponderadas mostró un **F1-score promedio ponderado de apenas 0.16** en ese caso, reflejando pobres desempeños fuera del acierto trivial. Igualmente, al excluir el ruido y tratar de clasificar solo las playlists con clusters HDBSCAN válidos (82 clases), el modelo XGBoost rendía prácticamente al azar (**3.7% de accuracy**), lo cual es consistente con 82 clases balanceadas sin la opción de predecir la mayoritaria. Estos resultados destacan la **limitación**

clave: la tarea de predecir el cluster real de una playlist resultó extremadamente difícil con los atributos disponibles. La alta dimensionalidad de clases y la distribución altamente desbalanceada (un cluster abarca ~40% de los datos, muchos clusters con <1% cada uno) provocaron que los modelos aprendieran una estrategia naïve pero efectiva en términos de accuracy: adivinar siempre la clase más común.

En resumen, el modelo final basado en HDBSCAN presentó un desempeño de clasificación muy limitado. Si bien logró *accuracy* de ~41%, esto se debió enteramente a explotar la predominancia del cluster de ruido, **fallando en diferenciar los clusters significativos** identificados por HDBSCAN. Esto indica que, a pesar de que HDBSCAN proporcionó una segmentación detallada, **las características utilizadas no fueron suficientes para predecir a qué cluster pertenece una playlist** más allá de distinguir las “atípicas” de las demás. Posibles razones incluyen: considerable solapamiento entre clusters en el espacio de features agregados, insuficiencia de los atributos para capturar la esencia distintiva de cada tipo de playlist, y el gran desequilibrio en el tamaño de clusters que sesgó el aprendizaje. El intento de agrupar clusters (en 7 categorías híbridas) mejoró ligeramente el balance, elevando la accuracy a ~30%, pero el problema fundamental persistió – el modelo seguía sesgado a la clase mayor (30% de las playlists). En conclusión, si el objetivo es predecir correctamente la pertenencia de una playlist a un cluster de afinidad, **serían necesarias estrategias adicionales** (p. ej., reducir la granularidad de clusters, seleccionar características más discriminativas, o aplicar técnicas de re-muestreo/ponderación para el desbalance) dado que el enfoque actual produce muchos errores. Los errores se concentran en confundir prácticamente todas las playlists no triviales, lo cual limita la utilidad directa de este modelo de predicción en aplicaciones como recomendaciones automatizadas basadas en estos clusters.

IV. Conclusiones.

En conclusión, el pipeline desarrollado – desde la exploración y limpieza de datos hasta la extracción de patrones frecuentes y el agrupamiento – proporciona un **informe integral del conjunto de playlists** y un mecanismo interpretable para **predecir canciones faltantes**. Técnicamente, integró métodos de minería de datos (reglas de asociación) con aprendizaje no supervisado (clustering) y esbozó componentes de aprendizaje supervisado (clasificación) y secuencial, demostrando un enfoque multidisciplinario típico de *Recomendadores de música*. Este trabajo sienta las bases para futuras mejoras, como incorporar datos de muchos más usuarios/playlists para robustecer los patrones, o entrenar el modelo secuencial de *next-track* para capturar transiciones musicales. No obstante, incluso con los datos dados, los resultados muestran que es posible inferir con buen criterio las canciones ausentes en playlists basándose en el **perfil sonoro y temático** de cada lista y en **relaciones históricas entre canciones** aprendidas del conjunto. En ámbitos aplicados, este enfoque ayudaría a plataformas de *streaming* a autocompletar playlists de usuarios manteniendo la cohesión musical, mejorando así la experiencia de descubrimiento de canciones de forma transparente y basada en datos.

V. Adjuntos.

TABLA I
DESCRIPCIÓN DE VARIABLES NIVEL PLAYLIST

Atributo	Tipo	Descripción
pid	Entero	ID de la playlist

name	Texto	Nombre de la playlist (opcional)
num_holdouts	Entero	Canciones ocultas
num_samples	Entero	Canciones visibles
num_tracks	Entero	Total de canciones
tracks	Lista	Lista de canciones visibles

Adicionalmente, para enriquecer el dataset se agregan datos provenientes de MusicBrainz y AcousticBrainz, donde hay información adicional sobre las canciones en forma de descripciones acústicas y metadatos generados automáticamente a partir del análisis de las señales musicales. Estos serán atributos adicionales que buscan capturar elementos musicales como género, estado de ánimo, ritmo y timbre.. A cada canción registrada en los playlist del conjunto de datos original se le asocian los nuevos datos, que consisten en 22 variables adicionales. El dataset puede utilizarse para tareas de minería de datos como clasificación de géneros, detección de estados de ánimo, análisis de estilos musicales y segmentación de audiencias.

TABLA II
DESCRIPCIÓN DE VARIABLES NIVEL CANCIÓN (DATASET ORIGINAL DEL CHALLENGE)

Atributo	Tipo	Rango Estimado	Descripción
pos	Entero	0 - 99	Posición en la playlist
track_name	Texto	-	Nombre de la canción
track_uri	Texto	22 caracteres	URI único en Spotify
artist_name	Texto	-	Artista principal
artist_uri	Texto	22 caracteres	URI del artista
album_name	Texto	-	Nombre del álbum
album_uri	Texto	22 caracteres	URI del álbum
duration_ms	Entero	30.000 - 600.000 ms	Duración de la canción

TABLA III
DESCRIPCIÓN DE VARIABLES NIVEL CANCIÓN (MUSICBRAINZ)

Atributo	Tipo	Descripción
mbid	Texto	MusicBrainz ID único para la canción
genre_mb	Texto	Género según MusicBrainz
bpm	Real	Tempo promedio en beats por minuto
energy	Real	Intensidad percibida de la canción (escala relativa)
danceability_ll	Real	Medida estimada de bailabilidad basada en modelo de regresión logística
loudness	Real	Nivel relativo de volumen
rating_value	Entero	Calificación media

rating_votes	Entero	Número de votos asociados
--------------	--------	---------------------------

TABLA IV
 DESCRIPCIÓN DE VARIABLES NIVEL CANCIÓN
 (ACOUSTICBRAINZ, HIGH LEVEL)

Atributo	Tipo	Valores posibles	Descripción
danceability	Categorico Probabilístico	danceable, not_danceable	Indica si la pista resulta apta para bailar, basándose en ritmo, pulso y regularidad del compás.
gender	Categorico Probabilístico	female, male	Género percibido de las voces principales en la grabación (femenina o masculina).
genre_dortmund	Categorico Probabilístico	alternative, blues, electronic, folkcountry, funksoulrnb, jazz, pop, raphiphop, rock	Asigna la canción a uno de los grandes géneros musicales entrenados en el modelo de Dortmund.
genre_electronic	Categorico Probabilístico	ambient, dnb, house, techno, trance	Clasifica subgéneros dentro de la música electrónica según el modelo especializado.
genre_rosamerica	Categorico Probabilístico	cla (Classic), dan(dance), hip(hiphop), jaz(jazz), pop, rhy(rhythm and blues), roc(rock), spe (spoken word)	Género estimado por el modelo Rosamerica; cubre estilos que van de clásica hasta spoken word.
ismir04_rhythm	Categorico Probabilístico	ChaChaCha, Jive, Quickstep, Rumba-American, Rumba-Misc, Samba, Tango, VienneseWaltz, Waltz	Identifica el ritmo/danza dominante según la taxonomía usada en ISMIR 2004.
genre_tzanetakis	Categorico Probabilístico	"blu": blues, "cla": classical (clásica), "cou": country, "dis": disco, "hip": hip hop, "jaz": jazz, "met": metal, "pop": pop, "reg": reggae, "roc": rock	Clasificación de género basada en el conjunto GTZAN de Tzanetakis; cubre los 10 géneros clásicos.

mood_acoustic	Categorico Probabilístico	acoustic, not_acoustic	Indica si la pista suena predominantemente acústica (instrumentos no electrónicos).
mood_aggressive	Categorico Probabilístico	aggressive, not_aggressive	Mide la energía y la agresividad percibida en la interpretación y producción.
mood_electronic	Categorico Probabilístico	electronic, not_electronic	Señala la presencia dominante de texturas y elementos de producción electrónica.
mood_happy	Categorico Probabilístico	happy, not_happy	Evalúa la valencia emocional de la pista, si transmite una sensación alegre.
mood_party	Categorico Probabilístico	party, not_party	Determina si la pista es adecuada para un ambiente de fiesta o club.
mood_relaxed	Categorico Probabilístico	relaxed, not_relaxed	Indica si la pista tiene un tempo y atmósfera tranquilos y relajantes.
mood_sad	Categorico Probabilístico	sad, not_sad	Mide la carga emocional melancólica o triste de la canción.
moods_mirex	Categorico Probabilístico	Cluster1 - Cluster5	Agrupar la canción en uno de cinco clusters de "estado de ánimo" definidos en la tarea MIREX de 2007.
tonal_atonal	Categorico Probabilístico	atonal, tonal	Clasifica la música según la presencia de una tonalidad clara (tonal) o ausencia de estructura tonal (atonal).
voice_instrumental	Categorico Probabilístico	instrumental, voice	Indica si predominan instrumentos sin canto (instrumental) o si hay voces destacadas (voice).

timbre	Categorico Probabilistico	bright, dark	Describe la cualidad sonora general: “brillante” (agudos y presencias) o “oscura” (graves y resonancias).
--------	-------------------------------	--------------	--

Los datos de los atributos expuestos en la tabla IV están disponibles en forma de una matriz de probabilidad: cada atributo tiene un diccionario donde se listan las probabilidades de cada categoría. Frente a todo el conjunto extendido de datos que se tiene se puede decir que se tiene una enorme dimensionalidad: 23 atributos por canción y 6 por playlist. Incluso se puede llegar a considerar variable ya que cada playlist puede albergar un número distinto de canciones.

IV. Bibliografía.

1. **Han, J., Pei, J., & Yin, Y. (2000).** *Mining Frequent Patterns without Candidate Generation*. In ACM SIGMOD International Conference on Management of Data (SIGMOD'00), 1–12. ([cs.sfu.ca, dl.acm.org](http://cs.sfu.ca/dl.acm.org))
– Presenta el algoritmo FP-Growth, mucho más eficiente que Apriori para grandes conjuntos de datos.
2. **Han, J., Kamber, M., & Pei, J. (2011).** *Data Mining: Concepts and Techniques* (3ª ed.). Morgan Kaufmann. (myweb.sabanciuniv.edu)
– Manual de referencia en minería de datos que cubre EDA, preprocesamiento, minería de patrones, clustering y sistemas de recomendación.
3. **McInnes, L., Healy, J., & Saul, N. (2018).** *UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction*. Journal of Open Source Software, 3(29), 861. (joss.theoj.org, [arXiv](https://arxiv.org/abs/1802.03426))
– Introduce UMAP, técnica no lineal para proyección y visualización de datos en dos dimensiones.
4. **Shlens, J. (2014).** *A Tutorial on Principal Component Analysis*. arXiv:1404.1100. ([arXiv](https://arxiv.org/abs/1404.1100))
– Tutorial accesible que expone la intuición y matemáticas detrás de PCA paso a paso.
5. **Apriori algorithm.** *Wikipedia*. Última consulta: “The Apriori algorithm was proposed by Agrawal and Srikant in 1994.” ([Wikipedia](https://en.wikipedia.org/wiki/Apriori_algorithm))
– Visión general y enlaces a implementaciones de Apriori en múltiples librerías de minería de datos.